DATABASE DESIGN DELIVERABLE

University Student Management System

Database Design Documentation — Entities, Relationships, ER Diagram & Relational Schema

TEAM	TEAM LEADER	DATABASE	DATE
Zomoroda (10 members)	Yara Yasser Mostafa	Microsoft SQL Server	12 October 2026

CONTENTS

1. Introduction & Scope
2. Entities
3. Relationships
4. Entity–Relationship Diagram
5. Tables
6. Relational Schema

1. Introduction & Scope

This document describes the database design of the University Student Management System. It identifies the entities and the reason each one is modelled as an entity, defines every relationship with its cardinality and participation, presents the full Entity–Relationship Diagram, and maps the design to the tables and the relational schema that will be implemented in Microsoft SQL Server.

25	38	28	3
Entities	Relationships	Tables	Tables from M:N relationships

Central table: Enrollment links a student to a course offering. Attendance, exam seats and grades all depend on it, so a student appears in a roster, an exam hall or a result sheet only if enrolled in that offering.

DESIGN SCOPE

- Real data is stored for the Faculty of Computers & AI; the other colleges appear in the interface only.
- One university, so University is a constant, not a table.
- Derived values (GPA, totals, letters, attendance %, seats left, registration status) are calculated, not stored.

NOTATION

- **ER diagram:** Chen notation (rectangles, diamonds, ISA circles).
- **Relational schema:** crow’s foot notation.
- **Cardinality** is written in the order of the relationship sentence, e.g. 1 : N.
- **Total** participation means every instance must take part.

2. Entities

25 entities

Each entity represents something the system must store several facts about and has many instances. Entities are grouped by area; the primary key shown is the key of the resulting table.

ORGANISATION

No.	Entity	Type	Primary key	Why it is an entity
1	**College**	Strong	`CollegeCode`	Core organisational unit (five colleges). Each has its own code, dean, status and academic rules, and owns departments, staff, students and courses. Rule values (level thresholds, attendance limits) are single-valued per college, so they are attributes of College.
2	**CreditLoadRule**	Weak · owner College	`RuleID`	Multivalued attribute of College: each college defines several GPA bands, each with a maximum credit load. A band has no meaning without its college.
3	**Department**	Strong	`DepartmentID`	A college has many departments, each with a code, name, head and status. Students, staff and courses are assigned to a department.
4	**Room**	Strong	`RoomCode`	Needed to detect room clashes and to check capacity for lectures, groups and exams. Storing a room as free text would make these checks impossible.

PEOPLE

No.	Entity	Type	Primary key	Why it is an entity
5	**Person**	Superclass	`PersonID`	Holds the personal data shared by students, staff and admins (names, national ID, contact details), so it is stored once instead of being repeated in every role table.
6	**UserAccount**	Strong	`LoginID`	Separates sign-in credentials from personal data: not every person signs in (e.g. teaching assistants), and passwords are kept in their own table.
7	**Student**	Subclass of Person	`StudentID`	Student-specific data: department, level, academic advisor, academic status, guardian details and previous qualification.
8	**Staff**	Subclass of Person	`StaffID`	Instructors and teaching assistants share the same attributes (rank, department, office), so they form one entity with a StaffType attribute.
9	**Admin**	Subclass of Person · Superclass	`AdminID`	Common part of both administrator types; AdminType records which specialisation the admin belongs to.
10	**UniversityAdmin**	Subclass of Admin	`AdminID`	Manages colleges, college admins and university-wide permissions; not tied to any college.
11	**CollegeAdmin**	Subclass of Admin	`AdminID`	Manages exactly one college and approves and publishes its results and schedules. Only one admin per college can be active.

CATALOGUE & REGISTRATION

No.	Entity	Type	Primary key	Why it is an entity
12	**AcademicTerm**	Strong	`TermID`	Schedules, registration, attendance, exams and results repeat every term, so they are all linked to a term.
13	**Course**	Strong	`CourseID`	Catalogue entry (code, name, level, credit hours, activity type) that stays the same across terms.
14	**CourseOffering**	Strong	`OfferingID`	A course taught in a specific term with its instructor, lecture time and room. It also holds the result workflow status, because each offering has exactly one result sheet.
15	**ClassGroup**	Weak · owner CourseOffering	`GroupID`	Section or lab groups of an offering, each with a TA, time, room and seat capacity. Identified by the offering plus the group number.
16	**SchedulePublication**	Weak · owners College, AcademicTerm	`CollegeCode, TermID, Level`	The lecture schedule is published per college, term and level, with its own status and publish time.
17	**RegistrationPeriod**	Weak · owners College, AcademicTerm	`CollegeCode, TermID`	Registration opening and closing dates and the manual override, defined for each college in each term.
18	**RegistrationForm**	Strong	`FormID`	A student's saved registration for one term. The advisor reviews it, so the decision, note and date must be stored.
19	**Enrollment**	Associative	`EnrollmentID`	Resolves the M:N relationship between Student and CourseOffering. It has its own attributes (registration date, chosen group, five grade components) and attendance and exam seats depend on it.

TEACHING, EXAMS & RESULTS

No.	Entity	Type	Primary key	Why it is an entity
20	**ClassSession**	Weak · owner CourseOffering	`SessionID`	Each actual lecture or group meeting on a given date; attendance is recorded per session.
21	**Exam**	Weak · owner CourseOffering	`ExamID`	The midterm and final of each offering, with date, time, room, committee name and committee head.
22	**ExamPublication**	Weak · owners College, AcademicTerm	`CollegeCode, TermID, ExamType`	Midterm and final timetables are published separately for each college and term, each with its own status.
23	**GradeScale**	Strong · lookup	`Letter`	Letter thresholds and grade points kept as data, so they can change without changing code. It has no relationship because the letter is derived from the total by range comparison.

PERMISSIONS & NOTIFICATIONS

No.	Entity	Type	Primary key	Why it is an entity
24	**Permission**	Strong	`PermissionKey`	Each role permission with its key, default value, locked flag and current value; applies to the whole university.

No.	Entity	Type	Primary key	Why it is an entity
25	**Notification**	Strong	`NotificationID`	Messages sent to a user (e.g. results published, advisor requested changes) with a read status.

3. Relationships

38 relationships

Read each relationship as a sentence: first entity, verb, second entity. Cardinality follows the same order. Identifying relationships connect a weak entity to its owner.

ORGANISATION

No.	Relationship	Cardinality	Participation	Justification
R1	`College` *contains* `Department`	`1 : N`	College: Partial Department: Total	Each department belongs to exactly one college; a college has many departments.
R2	`Staff` *heads* `Department`	`1 : 1`	Staff: Partial Department: Partial	A department has at most one head, and a staff member heads at most one department.
R3	`Staff` *is dean of* `College`	`1 : 1`	Staff: Partial College: Partial	Each college has one dean, who leads only that college.
R4	`College` *owns* `Room`	`1 : N`	College: Partial Room: Partial	A room belongs to one college or is shared between colleges (no owner).
R5	`College` *sets* `CreditLoadRule` identifying	`1 : N`	College: Total CreditLoadRule: Total	A college defines several GPA-based credit-load bands; a band exists only within its college (identifying).

PEOPLE

No.	Relationship	Cardinality	Participation	Justification
R6	`Person` *has* `UserAccount`	`1 : 1`	Person: Partial UserAccount: Total	A person who signs in has exactly one account; some people (e.g. TAs) have none.
R7	`Department` *enrolls* `Student`	`1 : N`	Department: Partial Student: Total	Each student belongs to one department; a department has many students.
R8	`Department` *employs* `Staff`	`1 : N`	Department: Partial Staff: Total	Each staff member belongs to one department; a department has many staff.
R9	`Staff` *advises* `Student`	`1 : N`	Staff: Partial Student: Partial	An advisor has many students; a student has at most one advisor. Graduated or inactive students have no advisor, while continuing students must have one. Not every instructor is an advisor.
R10	`College` *managed by* `CollegeAdmin`	`1 : N`	College: Partial CollegeAdmin: Total	A college admin manages one college. A college can have several admins over time, but only one active.
R11	`Person` *is a* `Student / Staff / Admin`	`ISA`	Person: Total Disjoint	Every person is exactly one of Student, Staff or Admin (total, disjoint). Shared personal data stays in Person.
R12	`Admin` *is a* `UniversityAdmin / CollegeAdmin`	`ISA`	Admin: Total Disjoint	Every admin is either a university admin or a college admin (total, disjoint).

CATALOGUE & REGISTRATION

No.	Relationship	Cardinality	Participation	Justification
R13	`Department` *offers* `Course`	`1 : N`	Department: Partial Course: Total	Each course belongs to one department; a department offers many courses.
R14	`Course` *requires* `Course`	`M : N`	Course: Partial (both roles)	A course can have several prerequisites and can be a prerequisite of several courses.
R15	`Course` *offered as* `CourseOffering`	`1 : N`	Course: Partial CourseOffering: Total	A course is offered in many terms; each offering is of one course.
R16	`AcademicTerm` *includes* `CourseOffering`	`1 : N`	AcademicTerm: Partial CourseOffering: Total	A term contains many offerings; a course is offered at most once per term.
R17	`Staff` *teaches* `CourseOffering`	`1 : N`	Staff: Partial CourseOffering: Total	Each offering has one weekly lecture taught by one instructor.
R18	`Room` *hosts* `CourseOffering`	`1 : N`	Room: Partial CourseOffering: Total	Each lecture is held in one room; a room hosts many lectures at different times.
R19	`CourseOffering` *has* `ClassGroup` identifying	`1 : N`	CourseOffering: Partial ClassGroup: Total	An offering is split into section or lab groups; a group cannot exist without its offering (identifying).
R20	`Staff` *assists* `ClassGroup`	`1 : N`	Staff: Partial ClassGroup: Total	Each group is taught by one teaching assistant.
R21	`Room` *hosts* `ClassGroup`	`1 : N`	Room: Partial ClassGroup: Total	Each group meets in one room or lab.
R22	`College` + `AcademicTerm` *publishes* `SchedulePublication` identifying	`1 : 1 : N`	College: Partial SchedulePublication: Total	The lecture schedule is published per college, term and level.
R23	`College` + `AcademicTerm` *opens* `RegistrationPeriod` identifying	`1 : 1 : N`	College: Partial RegistrationPeriod: Total	Each college has one registration period per term.
R24	`ClassGroup` *chosen in* `Enrollment`	`1 : N`	ClassGroup: Partial Enrollment: Partial	A student chooses one group per offering; empty when the course has no section or lab.
R25	`Student` + `AcademicTerm` *submits* `RegistrationForm`	`1 : 1 : N`	Student: Partial RegistrationForm: Total	A student saves one registration form per term.
R26	`Staff` *reviews* `RegistrationForm`	`1 : N`	Staff: Partial RegistrationForm: Partial	The advisor approves the form or requests changes; a form may not be reviewed yet.
R27	`ClassSession` *attended by* `Enrollment`	`M : N`	ClassSession: Partial Enrollment: Partial	Each session records attendance for many enrolled students, and each enrollment has many sessions; the status belongs to the pair.
R28	`Exam` *seats* `Enrollment`	`M : N`	Exam: Partial Enrollment: Partial	Every enrolled student receives a seat number in each exam of the course.

No.	Relationship	Cardinality	Participation	Justification
R29	CollegeAdmin *approves & publishes* CourseOffering	1 : N	CollegeAdmin: Partial CourseOffering: Partial	The college admin approves and publishes each offering's results; results may still be pending.
R30	Student *registers* CourseOffering via Enrollment	M : N	Student: Partial CourseOffering: Partial	A student registers many offerings and an offering has many students. The registration has its own attributes, so it is modelled as the associative entity Enrollment.

TEACHING, EXAMS & RESULTS

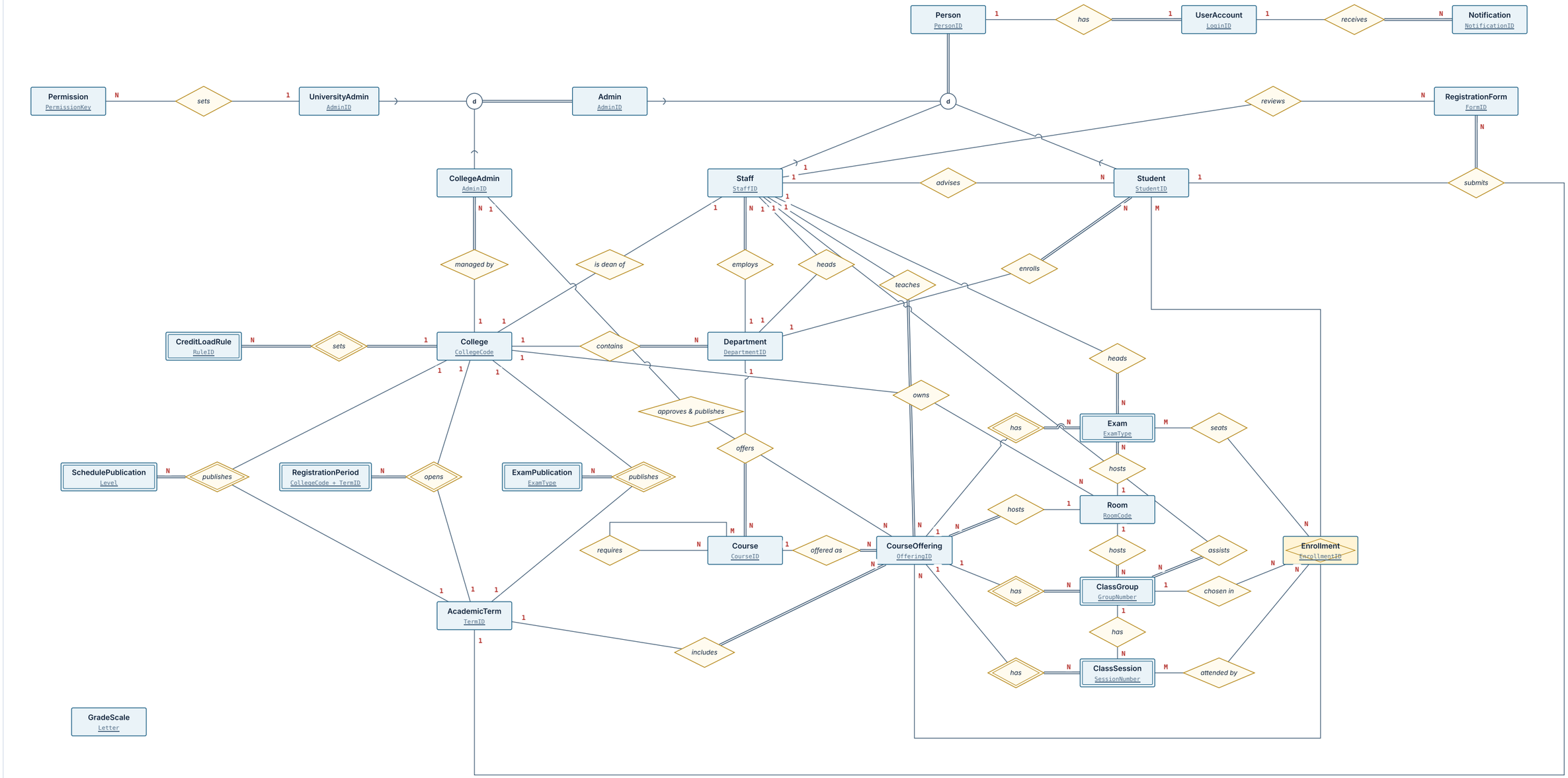
No.	Relationship	Cardinality	Participation	Justification
R31	CourseOffering *has* ClassSession identifying	1 : N	CourseOffering: Partial ClassSession: Total	Attendance is taken per meeting, so an offering has many dated sessions (identifying).
R32	ClassGroup *has* ClassSession	1 : N	ClassGroup: Partial ClassSession: Partial	A section or lab session belongs to one group; lecture sessions have no group.
R33	CourseOffering *has* Exam identifying	1 : N	CourseOffering: Partial Exam: Total	Each offering has at most one midterm and one final (identifying).
R34	Room *hosts* Exam	1 : N	Room: Partial Exam: Total	Each exam is held in one room.
R35	Staff *heads* Exam	1 : N	Staff: Partial Exam: Total	Each exam has one committee head.
R36	College + AcademicTerm *publishes* ExamPublication identifying	1 : 1 : N	College: Partial ExamPublication: Total	Midterm and final timetables are published separately per college and term.

PERMISSIONS & NOTIFICATIONS

No.	Relationship	Cardinality	Participation	Justification
R37	UniversityAdmin *sets* Permission	1 : N	UniversityAdmin: Partial Permission: Partial	The university admin switches permissions on or off for the whole university; the last editor is recorded.
R38	UserAccount *receives* Notification	1 : N	UserAccount: Partial Notification: Total	Each notification is sent to one user; a user receives many notifications.

4. Entity–Relationship Diagram

Chen notation



Entity · Weak entity · Relationship · Identifying relationship · Associative entity · d ISA (disjoint) · Total participation · Partial participation · Lines crossing without connecting

5. Tables

28 tables

Each strong entity becomes a table, each weak entity a table keyed through its owner, each M:N relationship a new table whose key combines both sides, each 1:N relationship a foreign key on the N side, and each subclass a table that shares its superclass key.

PK Primary key FK Foreign key UQ Unique NULL Column accepts NULL (all others are NOT NULL)

→ Table Referenced table

College

From entity

PK	**CollegeCode**	NVARCHAR(10)	FCAI, EDU, ALS, SCI, TH
	NameEn	NVARCHAR(150)	
	NameAr	NVARCHAR(150)	
	EstablishedYear	SMALLINT	
FK UQ	DeanStaffID	INT NULL	→ Staff Dean of the college
	LogoUrl	NVARCHAR(300) NULL	
	PhotoUrl	NVARCHAR(300) NULL	
	Status	NVARCHAR(10)	Active / Inactive
	SophomoreCredits	TINYINT	Default 30
	JuniorCredits	TINYINT	Default 72
	SeniorCredits	TINYINT	Default 108
	GraduationCredits	TINYINT	Default 132
	AttendanceWarningPct	TINYINT	Default 75
	ExamDeprivationPct	TINYINT	Default 65

CreditLoadRule

WEAK

From entity

PK	**RuleID**	INT	
FK	CollegeCode	NVARCHAR(10)	→ College
	MinGPA	DECIMAL(3,2)	
	MaxGPA	DECIMAL(3,2)	
	MaxCreditHours	TINYINT	12 / 18 / 21

Department

From entity

PK	**DepartmentID**	INT	
FK	CollegeCode	NVARCHAR(10)	→ College
	Code	NVARCHAR(10)	Unique within the college
	Name	NVARCHAR(150)	
FK UQ	HeadStaffID	INT NULL	→ Staff Head of department
	Status	NVARCHAR(10)	Active / Inactive

Room

From entity

PK	**RoomCode**	NVARCHAR(30)	CS-118, Main Hall 2
	Building	NVARCHAR(100)	
	Type	NVARCHAR(20)	Lecture hall / Classroom / Lab / Exam hall
	Capacity	SMALLINT	
FK	CollegeCode	NVARCHAR(10) NULL	→ **College** NULL = shared room

Person

From entity

PK	**PersonID**	INT	
	NameEn	NVARCHAR(150)	
	NameAr	NVARCHAR(150)	
	Gender	NVARCHAR(10)	
	Nationality	NVARCHAR(50)	
UQ	NationalID	NVARCHAR(14)	
	DateOfBirth	DATE	
	PlaceOfBirth	NVARCHAR(100) NULL	
	Mobile	NVARCHAR(20)	Editable by the user
	Address	NVARCHAR(250) NULL	
	EmergencyName	NVARCHAR(150) NULL	
	EmergencyPhone	NVARCHAR(20) NULL	

UserAccount

From entity

PK	**LoginID**	NVARCHAR(20)	ST-... / INS-... / ADM-...
FK UQ	PersonID	INT	→ **Person**
UQ	Email	NVARCHAR(150)	first.last@hurghada.edu
	PasswordHash	NVARCHAR(255)	
	Role	NVARCHAR(20)	Student / Instructor / CollegeAdmin / UniversityAdmin
	Status	NVARCHAR(10)	Active / Inactive
	LastSignInAt	DATETIME2 NULL	
	CreatedAt	DATETIME2	

Student

From entity

PK FK	**StudentID**	INT	→ **Person** Same ID as in Person
FK	DepartmentID	INT	→ **Department** Department or specialisation
FK	AdvisorStaffID	INT NULL	→ **Staff** Academic advisor (NULL if graduated or inactive)
	Level	NVARCHAR(10)	Freshman / Sophomore / Junior / Senior
	EnrollmentDate	DATE	
	AcademicStatus	NVARCHAR(15)	Continuing / Graduated / Suspended / Withdrawn
	GuardianName	NVARCHAR(150)	
	GuardianOccupation	NVARCHAR(100) NULL	
	GuardianMobile	NVARCHAR(20)	
	City	NVARCHAR(50)	
	HomePhone	NVARCHAR(20) NULL	
	CertificateType	NVARCHAR(50)	
	SchoolName	NVARCHAR(150)	
	GraduationYear	SMALLINT	
	PreviousScore	DECIMAL(5,2)	

Staff

From entity

PK FK	**StaffID**	INT	→ **Person** Same ID as in Person
FK	DepartmentID	INT	→ **Department**
	StaffType	NVARCHAR(20)	Instructor / TeachingAssistant
	AcademicRank	NVARCHAR(30)	Professor ... TA
	Specialization	NVARCHAR(100) NULL	
	Office	NVARCHAR(50) NULL	
	Status	NVARCHAR(10)	Active / Inactive

Admin

From entity

PK FK	**AdminID**	INT	→ **Person** Same ID as in Person
	AdminType	NVARCHAR(10)	University / College

UniversityAdmin

From entity

PK FK	**AdminID**	INT	→ **Admin** Same ID as in Admin

CollegeAdmin

From entity

PK FK	**AdminID**	INT	→ **Admin** Same ID as in Admin
FK	CollegeCode	NVARCHAR(10)	→ **College** College managed
	IsActive	BIT	One active admin per college
	AssignedAt	DATE	

AcademicTerm

From entity

PK	**TermID**	INT	
	AcademicYear	NVARCHAR(9)	2026/2027
	TermName	NVARCHAR(10)	First / Second / Summer
	StartDate	DATE	
	EndDate	DATE	
	IsCurrent	BIT	Only one current term

Course

From entity

PK	**CourseID**	INT	
FK	DepartmentID	INT	→ **Department**
	Code	NVARCHAR(10)	e.g. IS 301 · unique within the department
	Name	NVARCHAR(150)	
	Level	NVARCHAR(10)	
	CreditHours	TINYINT	
	ActivityType	NVARCHAR(10)	Section / Lab / None
	Status	NVARCHAR(10)	Active / Inactive

CourseOffering

From entity

PK	**OfferingID**	INT	
FK	CourseID	INT	→ **Course**
FK	TermID	INT	→ **AcademicTerm** Unique with CourseID
FK	InstructorStaffID	INT	→ **Staff** Lecturer
	LectureDay	NVARCHAR(3)	Sat ... Thu
	LectureStart	TIME	End = start + credit hours
FK	LectureRoomCode	NVARCHAR(30)	→ **Room**
	ResultStatus	NVARCHAR(10)	Draft / Submitted / Approved / Returned / Published
	SubmittedAt	DATETIME2 NULL	
	ReturnReason	NVARCHAR(300) NULL	Required when Returned
FK	ApprovedByAdminID	INT NULL	→ **CollegeAdmin**
	ApprovedAt	DATETIME2 NULL	
FK	PublishedByAdminID	INT NULL	→ **CollegeAdmin**
	PublishedAt	DATETIME2 NULL	

ClassGroup

WEAK

From entity

PK	**GroupID**	INT	
FK	OfferingID	INT	→ **CourseOffering**
	GroupNumber	TINYINT	Unique within the offering
	GroupType	NVARCHAR(10)	Section / Lab
FK	TAStaffID	INT	→ **Staff** Teaching assistant
	Day	NVARCHAR(3)	
	StartTime	TIME	Duration 2 hours
FK	RoomCode	NVARCHAR(30)	→ **Room**
	SeatCapacity	SMALLINT	

SchedulePublication

WEAK

From entity

PK FK	**CollegeCode**	NVARCHAR(10)	→ **College**
PK FK	**TermID**	INT	→ **AcademicTerm**
PK	**Level**	NVARCHAR(10)	
	Status	NVARCHAR(20)	Draft / Published / UnpublishedChanges
	PublishedAt	DATETIME2 NULL	

RegistrationPeriod

WEAK

From entity

PK FK	**CollegeCode**	NVARCHAR(10)	→ **College**
PK FK	**TermID**	INT	→ **AcademicTerm**
	RegistrationOpens	DATETIME2	
	RegistrationCloses	DATETIME2	
	ManualOverride	NVARCHAR(15)	None / ForcedClosed / ForcedOpen

RegistrationForm

From entity

PK	**FormID**	INT	
FK	StudentID	INT	→ **Student**
FK	TermID	INT	→ **AcademicTerm** Unique with StudentID
	SavedAt	DATETIME2	
FK	AdvisorStaffID	INT NULL	→ **Staff** Reviewing advisor
	Decision	NVARCHAR(20)	AwaitingSignature / Approved / ChangesRequested
	AdvisorNote	NVARCHAR(500) NULL	Required when ChangesRequested
	DecidedAt	DATETIME2 NULL	

Enrollment — ASSOCIATIVE

From entity

PK	**EnrollmentID**	INT	
FK	StudentID	INT	→ **Student**
FK	OfferingID	INT	→ **CourseOffering** Unique with StudentID
FK	GroupID	INT NULL	→ **ClassGroup** NULL if the course has no section or lab
	RegisteredAt	DATETIME2	
	AttendanceMark	DECIMAL(4,1) NULL	Out of 10
	PracticalMark	DECIMAL(4,1) NULL	Out of 20
	MidtermMark	DECIMAL(4,1) NULL	Out of 30
	OralMark	DECIMAL(4,1) NULL	Out of 10
	FinalMark	DECIMAL(4,1) NULL	Out of 30

ClassSession — WEAK

From entity

PK	**SessionID**	INT	
FK	OfferingID	INT	→ **CourseOffering**
FK	GroupID	INT NULL	→ **ClassGroup** NULL = lecture session
	SessionNumber	TINYINT	
	SessionDate	DATE	

Exam — WEAK

From entity

PK	**ExamID**	INT	
FK	OfferingID	INT	→ **CourseOffering**
	ExamType	NVARCHAR(10)	Midterm / Final · unique with OfferingID
	ExamDate	DATE	
	StartTime	TIME	
	EndTime	TIME	
FK	RoomCode	NVARCHAR(30)	→ **Room**
	CommitteeName	NVARCHAR(100)	
FK	HeadStaffID	INT	→ **Staff** Committee head

ExamPublication — WEAK

From entity

PK FK	**CollegeCode**	NVARCHAR(10)	→ **College**
PK FK	**TermID**	INT	→ **AcademicTerm**
PK	**ExamType**	NVARCHAR(10)	
	Status	NVARCHAR(20)	Draft / Published / UnpublishedChanges
	PublishedAt	DATETIME2 NULL	

GradeScale

From entity

PK	**Letter**	NVARCHAR(2)	A, A−, B+, B, C+, C, F
	MinTotal	TINYINT	90, 85, 80, 75, 70, 60, 0
	GradePoints	DECIMAL(3,2)	To be confirmed

Permission

From entity

PK	**PermissionKey**	NVARCHAR(60)	student.downloadTranscript
	Role	NVARCHAR(20)	Student / Instructor / CollegeAdmin
	GroupName	NVARCHAR(30)	
	Label	NVARCHAR(100)	
	Description	NVARCHAR(300) NULL	
	DefaultOn	BIT	
	IsLocked	BIT	Required = always on
	IsEnabled	BIT	
FK	UpdatedByAdminID	INT NULL	→ **UniversityAdmin** Last university admin who changed it
	UpdatedAt	DATETIME2 NULL	

Notification

From entity

PK	**NotificationID**	INT	
FK	LoginID	NVARCHAR(20)	→ **UserAccount**
	Title	NVARCHAR(150)	
	Body	NVARCHAR(500)	
	IsRead	BIT	
	CreatedAt	DATETIME2	

CoursePrerequisite

M:N

From relationship: Course requires Course (M:N)

PK FK	**CourseID**	INT	→ **Course**
PK FK	**PrerequisiteCourseID**	INT	→ **Course** Course required first

Attendance

M:N

From relationship: ClassSession attended by Enrollment (M:N)

PK FK	**SessionID**	INT	→ **ClassSession**
PK FK	**EnrollmentID**	INT	→ **Enrollment**
	Status	NVARCHAR(10)	Present / Late / Absent
FK	RecordedByStaffID	INT	→ **Staff** Instructor who recorded it
	RecordedAt	DATETIME2	

ExamSeat

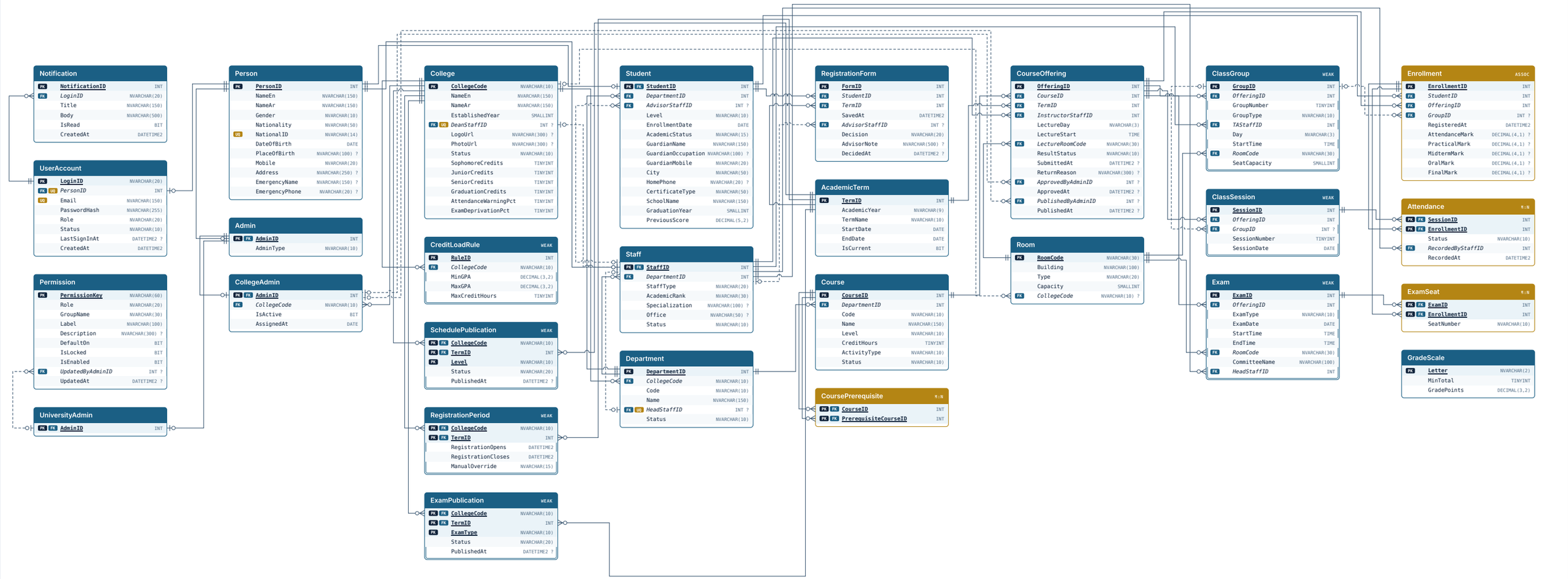
M:N

From relationship: Exam seats Enrollment (M:N)

PK FK	**ExamID**	INT	→ **Exam**
PK FK	**EnrollmentID**	INT	→ **Enrollment**
	SeatNumber	NVARCHAR(10)	Unique within the exam

6. Relational Schema

Crow's foot notation



Exactly one · Zero or one · Zero or many · Optional foreign key (NULL allowed) · Table from an entity · Table from a relationship (M:N or associative)